

Jukebox

Sandra, Júlía, Jenna, Sandra

Due 27 Feb by midday

Implement the simulation of a fair and democratic Jukebox in the Go language.

Assume that there is a party and the guests have a varied taste in music. The host wants to make sure that each guest will dance to some of the songs they like. Every guest may submit votes for their songs to the juke box. The songs should be maintained in a priority queue. The song with the most votes should be played next, but no song may be played more than once every thirty minutes and every song that received a vote must be played.

Develop a data structure to maintain the songs and their votes. and when they were played last. One go routine is responsible for playing the next song. If there is no song with a vote, a random song that was not played for less than thirty minutes shall be chosen.

Write go routines for the guests that submit votes to the jukebox. Use semaphores for coordination and shared storage for communication (no channels).

For testing assume that a guest submits a vote for only one song. Assume that guests submit votes with a individual rate λ . A rate of 0.5 means that a vote is submitted every 2 seconds on average while a rate of 2 means that a vote is submitted twice per second. Sample the delays from an exponential distribution with rate λ (you can use the rand package and determine the wait time by `ExpFloat64()/λ`

Show and explain your code and show a simulation with at least 12 guests.

Documentation

Print the help

```
$ ./main -h
```

Start the program with the default number of guests (12) and real time enabled. Real time means that the Jukebox thread will sleep for the same amount of time as the duration of the song that was selected.

```
$ ./main
```

Start the program with 30 guests and real time enabled.

```
$ ./main -guests=30
```

Start the program with the default number of guests (12) and real time off. If real time is off, then the Jukebox thread will only wait for four seconds between choosing songs.

```
$ ./main -realTime=false
```

Explanation of our solution

Our solution is concurrent. Voting happens concurrently with the jukebox playing songs.

In our solution, a “song” is a record with name, duration, votes, time of last play, and a timestamp “FirstVote” recording the last time that the vote count changed from 0 to 1. A song is initialized with a name, 0 votes, the duration of the song and FirstVote and LastPlayed get set to the nil value of Time (time.Time{}). Doing so ensures that we have a time value to compare to when picking songs.

```
type Song struct {
    Name string
    Votes int
    Duration time.Duration
    FirstVote time.Time
    LastPlayed time.Time
    Lock sync.RWMutex
}
```

Our songs are stored in a priority queue type called SongQueue which sorts the songs in decreasing number of votes.

```
type SongQueue struct {
    Songs []song.Song
    Lock sync.RWMutex
}
```

The song queue is shared memory. Mutual exclusion is ensured using read and write locks from Go’s sync package.

Each Guest is a goroutine running an infinite loop. Each guest has one favourite song which they always vote for. The Guest routines are initialised with a favourite song (chosen randomly from the song list) and lambda (guessing rate in inverse seconds, chosen randomly from a uniform distribution between 0.2 and 3.2). A guest routine casts a vote for their favourite song by incrementing that song’s vote count (under mutual exclusion), then sleeps for an amount of time determined by sampling from an exponential (Poisson) distribution with rate *lambda*, where lambda is that Guest’s unique lambda value.

In addition, when a guest votes on a song, if the vote count for that song is zero, then the “time of first vote” property of that song must be updated to the current time. This is used to stop songs from “rotting” ie having at least one vote but never being played.

The Jukebox is a goroutine that runs an infinite loop. When the jukebox needs to select the next song, it needs to ensure that the following requirements are met:

1. All things being equal, the jukebox gives priority to songs with more votes.
2. The jukebox must not play songs that have been played in the last 30 minutes.
3. The jukebox must ensure that if a song receives a vote, it will be played eventually. (Songs must not “rot”).
4. If none of the above holds, a random song is selected for play.

We have satisfied these requirements with the following algorithm for choosing a song from the song queue:

```
Lock the song queue
Sort the song queue //(in decreasing order of votes)
int song_to_play := -1 // the index of a song in the priority queue
boolean found_song := true

for each song in songqueue:

    // Find the song with the most votes...
    //.. that has not been played in the last 30 minutes.
    if !found_song
        if (this song was played > 30 minutes ago)
            song_to_play = song
            found_song = true
        fi
    fi

    // Search the whole list for songs that are ‘rotting’
    if (song has at least one vote && first vote was cast >1 hour ago):
        song_to_play = song
        break
    fi
end for

if song_to_play == -1:
    // No song has been found, so play a random song.
    choose a random song from the queue
fi

Unlock the song queue
```

When a song gets selected, its vote count gets reset to zero and time of last played gets set to time.Now().

Appendix: Our Favourite Songs

Sandra Ósk's favourite songs

Nothing but thives - Amsterdam
A perfect circle - Doomed
Alt-j - In cold blood
Death from above - Freeze me
Arctic monkeys - r u mine

Jenna's favourite songs

Architects - Of dust and nations
Architects - In elegance
Architects - Gone with the wind
Architects - Castles in the air
Allegaeon - 1.618

Sandra Rós's favourite songs

Allegaeon - Gray Matter Mechanics
Allegaeon - Threshold of Perception
Allegaeon - Behold (God I Am)
Allegaeon - All Hail Science
Allegaeon - Of Mind and Matrix
Allegaeon - Proponent for Sentience III - The Extermination
Cattle Decapitation - Forced Gender Reassignment
Cattle Decapitation - Manufactured Extinct
DevilDriver - Pray for Villains
DevilDriver - Pure Sincerity
FIDLAR - Cocaïne
FIDLAR - Cheap Beer
FIDLAR - 40oz. On Repeat
Goatwhore - Baring Teeth for Revolt
Halestorm - Freak Like Me
Halestorm - Mz. Hyde
K.Flay - We Hate Everyone
Marduk - Frontschwein
Marduk - Wartheland
Marduk - 502
Marduk - Scorched Earth

Meshuggah - Bleed
Meshuggah - New Millenium Cyanide Christ
Meshuggah - Demiurge
Once Human - Eye of Chaos
Once Human - You Cunt
Opeth - The Drapery Falls
Opeth - Demon of the Fall
Opeth - Face of Melinda
.... so many more... this list would get too long.

Júlia's revised favourite songs with unnecessarily long titles

1. Some People Know All Too Well How Bad Licorice, Or Any Candy For That Matter, Can Taste When Having Laid Out In The Sun Too Long - And I Think I Just Ate Too Much by The Hives
2. My Cosmic Autumn Rebellion (The Inner Life As Blazing Sield Of Defiance and Optimisim as Celestial Spear of Action) by The Flaming Lips
3. You Probably Couldn't See Me For The Lights but You Were Looking Straight At Me by Arctic Monkeys
4. There Is A Good Reason These Tables Are Numbered, Honey, You Just Haven't Thought of it Yet by Panic! at the Disco
5. Several Species Of Small Furry Animals Gathered Together In A Cave and Grooving With A Pict by Pink Floyd
6. Our Lawyers Made Us Change the Name of This Song So We Wouldn't Get Sued, Fall Out Boy

Júlia's favourite songs with the word "song" in the title

- Song 2, Blur
- Swamp Song, Tool
- Immigrant Song, Led Zeppelin
- Adam's Song, blink-182
- The Anchor Song, Björk
- Camel Song, KoRn
- A Complicated Song, Weird Al
- The Death Song, Marilyn Manson
- Highway Song, System of a Down
- I Slept With Someone in Fall Out Boy and All I Got Was This Stupid Song Written About Me, Fall Out Boy
- The Last Song, All American Rejects
- The Last Song, Elton John
- The Last Song, Foo Fighters
- The Last Song, The Smashing Pumpkins
- The Last Song, Hillary Duff
- Love Song, ACDC
- Love Song, Alice in Chains
- Love Song, The Damned

- Love Song, KoRn
- Love Song, Madonna
- Love Song, Marilyn Manson
- Mosquito Song, Queens of the Stone Age
- Panic Song, Green Day
- Prison Song, System of a Down
- Sad Song, Oasis
- Our Lawyers Made Us Change the Name of This Song So We Wouldn't Get Sued, Fall Out Boy
 - Additional coolness, an album named Songs About Fucking, Big Black

